SB-3 CPR E 488 MP-3 Report

Nolan Eastburn, Conner Ohnesorge, Owen Parker, Jason Xie
Date: 4/9/2025

`launcher_fire.c` Makefile

This Makefile is configured to build both a Linux kernel module (`launcher_driver.ko`) and a user-space program (`launcher_fire`) using a cross-compiler for ARM architecture.

Cross-Compilation Setup

The Makefile uses the `CROSS_COMPILE` environment variable to specify the cross-compiler toolchain:

```
1   CC := $(CROSS_COMPILE)gcc
```

This is the core mechanism that enables ARM architecture targeting. The `CROSS_COMPILE` variable is expected to be set in the environment before running the make command (e.g., `CROSS_COMPILE=arm-linux-gnueabihf-` or similar ARM toolchain prefix). When expanded, it creates commands like `arm-linux-gnueabihf-gcc` that invoke the cross-compiler instead of the host system's native compiler.

Kernel Module Building

For the kernel module (`launcher_driver.ko`):

1. `obj-m += launcher_driver.o` tells the kernel build system which object files to build into modules.
2. `KDIR := ../linux/linux-xlnx/` points to the Xilinx Linux kernel source directory (often used for Zynq ARM platforms).
3. The main kernel build is triggered with:

   ```
   1   $(MAKE) -C $(KDIR) M=${shell pwd} modules
   ```

   This invokes the kernel's build system, which will use the cross-compiler settings defined in the kernel's configuration.

User-Space Program Building

For the user application (`launcher_fire`):

1. The application is built directly using the cross-compiler via:

```
$(BIN): $(SOURCES)
	$(CC) $@.c -o $@
```

   The `$(CC)` variable expands to the cross-compiler as defined earlier.

Clean Target

The clean target is thorough, removing both the kernel module files and the user-space binary:

```
clean:
  -$(MAKE) -C $(KDIR) M=${shell pwd} clean || true
  -rm $(BIN) || true
  -rm *.o *.ko *.mod.{c,o} modules.order Module.symvers || true
```

The `launcher_fire.c` code is a user-space application that communicates with the kernel module through a device node (`/dev/launcher0`), sending commands to control what seems to be a physical launcher device (likely a USB missile launcher or similar gadget).

This configuration works for ARM because:

1. It uses the ARM cross-compiler toolchain via `CROSS_COMPILE`
2. It builds against an ARM-targeted kernel source tree (Xilinx's Linux kernel)
3. The resulting binaries will be compatible with ARM systems, specifically a Xilinx Zynq platform.

Boot Process Analysis

U-Boot Initialization (Bootloader Phase)

```
U-Boot 2020.01 (Mar 14 2025 - 15:19:07 +0000)

CPU:   Zynq 7z020
Silicon: v3.1
Model: Zynq Zed Development Board
DRAM:  ECC disabled 512 MiB
Flash: 0 Bytes
```

```
NAND:  0 MiB
MMC:   mmc@e0100000: 0
```

This section shows U-Boot 2020.01 bootloader starting. It identifies the hardware as a Zynq 7z020 CPU on a Zed Development Board with 512MB of RAM (with ECC disabled). It's detecting storage devices, including an MMC (SD card) interface.

```
Loading Environment from SPI Flash... SF: Detected s25fl256s1 with page size 256 Bytes, erase size 64 KiB, total 32 MiB
*** Warning - bad CRC, using default environment
```

U-Boot is attempting to load environment variables from SPI Flash memory. It finds an s25fl256s1 flash chip (32MB total) but encounters a CRC error, so it falls back to default settings.

```
In:    serial@e0001000
Out:   serial@e0001000
Err:   serial@e0001000
Net:
ZYNQ GEM: e000b000, mdio bus e000b000, phyaddr 0, interface rgmii-id
```

Sets up the console I/O through a serial port (UART) and initializes the Gigabit Ethernet MAC (GEM).

Boot Image Loading

```
Hit any key to stop autoboot:  2  1  0
switch to partitions #0, OK
mmc0 is current device
Scanning mmc 0:1...
Found U-Boot script /boot.scr
2010 bytes read in 33 ms (58.6 KiB/s)
## Executing script at 03000000
11543076 bytes read in 656 ms (16.8 MiB/s)
```

U-Boot is performing autoboot countdown.

After no interruption, it scans the first partition of the SD card, finds a boot script, and executes it.

This script then loads the kernel and initial ramdisk.

```
## Loading kernel from FIT Image at 10000000 ...
```

```
   Using 'conf@system-top.dtb' configuration
   Verifying Hash Integrity ... OK
   Trying 'kernel@1' kernel subimage
     Description:  Linux kernel
     Type:         Kernel Image
     Compression:  uncompressed
     Data Start:   0x100000e8
     Data Size:    4325680 Bytes = 4.1 MiB
     Architecture: ARM
     OS:           Linux
     Load Address: 0x00200000
     Entry Point:  0x00200000
     Hash algo:    sha256
     Hash value:
16a76e92c611898f8057d865ef087705fef1aceff96e78675bc68784fd25ac76
   Verifying Hash Integrity ... sha256+ OK
```

U-Boot is loading the Linux kernel from a FIT (Flattened Image Tree) image.

It verifies the hash integrity of the kernel (4.1 MiB in size) to ensure it hasn't been corrupted.

```
## Loading ramdisk from FIT Image at 10000000 ...
   [Details about the ramdisk loading]
## Loading fdt from FIT Image at 10000000 ...
   [Details about the device tree loading]
```

Next, it loads the initial RAM disk (6.9 MiB) and the Flattened Device Tree (FDT) file that describes the hardware to the kernel.

Linux Kernel Startup

```
Starting kernel ...

Booting Linux on physical CPU 0x0
Linux version 5.4.0-xilinx-v2020.1 (oe-user@oe-host) (gcc version 9.2.0
(GCC)) #1 SMP PREEMPT Fri Mar 14 15:18:45 UTC 2025
CPU: ARMv7 Processor [413fc090] revision 0 (ARMv7), cr=18c5387d
```

The kernel begins executing. This shows Linux 5.4.0 specifically built for Xilinx hardware. It's running on an ARMv7 processor.

```
Memory policy: Data cache writealloc
cma: Reserved 16 MiB at 0x1f000000
percpu: Embedded 15 pages/cpu s31948 r8192 d21300 u61440
```

```
Built 1 zonelists, mobility grouping on.  Total pages: 129920
Kernel command line: console=ttyPS0,115200 earlycon root=/dev/ram0 rw
```

The kernel is setting up memory management policies and showing the command line parameters that were passed to it. It will use a serial console and boot from an initial RAM disk.

```
Memory: 484528K/524288K available (6144K kernel code, 217K rwdata, 1840K
rodata, 1024K init, 131K bss, 23376K reserved, 16384K cma-reserved, 0K
highmem)
```

Memory summary: out of 512MB (524288K) total RAM, about 484MB is available for use after accounting for kernel code, data, and reserved regions.

Hardware Detection and Initialization

```
rcu: Preemptible hierarchical RCU implementation.
[...]
smp: Bringing up secondary CPUs ...
CPU1: thread -1, cpu 1, socket 0, mpidr 80000001
CPU1: Spectre v2: using BPIALL workaround
smp: Brought up 1 node, 2 CPUs
```

The kernel is initializing the RCU (Read-Copy-Update) subsystem and bringing up multiple CPU cores.

It's a dual-core system with Spectre vulnerability mitigations.

```
devtmpfs: initialized
VFP support v0.3: implementor 41 architecture 3 part 30 variant 9 rev 4
[...]
SCSI subsystem initialized
usbcore: registered new interface driver usbfs
usbcore: registered new interface driver hub
usbcore: registered new device driver usb
```

Initialization of various subsystems: device manager, floating-point support, SCSI, USB, etc.

File Systems and Network Setup

```
FPGA manager framework
Advanced Linux Sound Architecture Driver Initialized.
[...]
```

```
tcp_listen_portaddr_hash hash table entries: 512 (order: 0, 6144 bytes, linear)
[...]
Trying to unpack rootfs image as initramfs...
Freeing initrd memory: 7028K
```

Setting up FPGA management, sound drivers, TCP/IP networking stacks, and unpacking the initial root filesystem from RAM.

```
jffs2: version 2.2. (NAND) (SUMMARY)  © 2001-2006 Red Hat, Inc.
io scheduler mq-deadline registered
io scheduler kyber registered
[...]
spi-nor spi0.0: found s25fl256s1, expected n25q128a11
spi-nor spi0.0: s25fl256s1 (32768 Kbytes)
8 fixed-partitions partitions found on MTD device spi0.0
```

Setting up various filesystems, I/O schedulers, and detecting flash memory partitions. The system found a different SPI flash chip than expected but continues with it.

Device Detection and Driver Loading

```
Marvell 88E1510 e000b000.ethernet-ffffffff:00: attached PHY driver [Marvell 88E1510] (mii_bus:phy_addr=e000b000.ethernet-ffffffff:00, irq=POLL)
macb e000b000.ethernet eth0: Cadence GEM rev 0x00020118 at 0xe000b000 irq 26 (00:0a:35:00:1e:53)
[...]
usb usb1: New USB device found, idVendor=1d6b, idProduct=0002, bcdDevice= 5.04
usb usb1: New USB device strings: Mfr=3, Product=2, SerialNumber=1
usb usb1: Product: EHCI Host Controller
```

Detection and initialization of Ethernet and USB controllers with their respective drivers.

```
mmc0: SDHCI controller on e0100000.mmc [e0100000.mmc] using ADMA
[...]
mmc0: new high speed SDHC card at address 59b4
mmcblk0: mmc0:59b4 USD   14.7 GiB
mmcblk0: p1
```

SD card controller initialization. It detects a 14.7GB SDHC card with one partition.

Transition to Userspace

```
Freeing unused kernel memory: 1024K
Run /init as init process

INIT: version 2.88 booting

The kernel has completed initialization and is now starting the first
userspace process (/init), which is using SysVinit (version 2.88).

Starting udev
udevd[73]: starting version 3.2.8
[...]
FAT-fs (mmcblk0p1): Volume was not properly unmounted. Some data may be
corrupt. Please run fsck.
```

Starting the device manager (udev) and mounting filesystems. A warning appears about the FAT filesystem on the SD card.

```
Configuring packages on first boot....
 (This may take several minutes. Please do not power off the machine.)
[...]
INIT: Entering runlevel: 5
```

Running first-boot configurations and entering runlevel 5 (graphical multi-user mode).

Network and Service Configuration

```
Configuring network interfaces... udhcpc: started, v1.31.0
udhcpc: sending discover
[...]
udhcpc: no lease, forking to background
```

Attempting to configure network via DHCP, but it doesn't receive a lease (no DHCP server responding).

```
Starting Dropbear SSH server:
[...]
Public key portion is:
    ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAABAQDFi2F+hJ58qyEF5ZI0VNshOIuSYHRUfMaMcRfvd7yR/i
lXnshWpyT49fqkJ7ZiofJ2LtHc3i8+98yDtk3WWk9FFOiVFgum9rEiRh+lVimeRX1zv0AA+GZi
wQYmzFxxyPJgRxisuWOgZJ7VR8zZwdd/mizMBczpsTKv22QSx2ymgJUQQBnnr2fkeDZEhK34mh
1m+c/n+B0uLIvjBiy9SJeL38CVWsTzN0bmL26o2DKjwYTU+j//QWUC02r1kodxS4d9cr0GZyg9
```

```
    1/xtPHqk5+jVgbtTe2iapT0d+YFZFI/x4HkJSj7fp25qnGpc3hNczqUobnLy9KL0F4bpfOjwIc
    Gt root@avnet-digilent-zedboard-2020_1
```

Starting the SSH server (Dropbear) and generating SSH host keys.

```
1  Starting internet superserver: inetd.
2  Starting syslogd/klogd: done
3  Starting tcf-agent: OK
```

Starting various system services: internet services daemon, system logging, and TCF (Target Communication Framework) agent.

Kernel Messages For USB Device

> Note: This is prior to setup of the kernel object device driver.

```
1  avnet-digilent-zedboard-2020_1:~$ usb 1-1: new low-speed USB device number
   2 using ci_hdrc
2  usb 1-1: New USB device found, idVendor=2123, idProduct=1010, bcdDevice=
   0.01
3  usb 1-1: New USB device strings: Mfr=1, Product=2, SerialNumber=0
4  usb 1-1: Product: USB Missile Launcher
5  usb 1-1: Manufacturer: Syntek
6  hid-generic 0003:2123:1010.0001: device has no listeners, quitting
```

1. `usb 1-1: new low-speed USB device number 2 using ci_hdrc` - This indicates a new USB device connecting at the low-speed USB specification (1.5 Mbps) and is being assigned device number 2. The "ci_hdrc" refers to the USB host controller driver.
2. `usb 1-1: New USB device found, idVendor=2123, idProduct=1010, bcdDevice=0.01` - The system has identified the USB device with its vendor ID (2123) and product ID (1010). These unique identifiers tell the system what device is connected. The bcdDevice value (0.01) indicates the device's firmware/version number.
3. `usb 1-1: New USB device strings: Mfr=1, Product=2, SerialNumber=0` - This shows that the device provides manufacturer and product string descriptors but no serial number.
4. `usb 1-1: Product: USB Missile Launcher` - The product string identifies it as the USB Missile Launcher.
5. `usb 1-1: Manufacturer: Syntek` - The manufacturer is identified as Syntek.
6. `hid-generic 0003:2123:1010.0001: device has no listeners, quitting` - This is expected since the device needs a specific driver.

Changes made to `usb_skeleton.c`

Header and Configuration Changes

Added a new header inclusion: "launcher-commands.h" which contains device-specific constants and commands
Changed device identification:

Replaced generic vendor/product IDs with missile launcher specific IDs
Updated module table to use these missile launcher IDs

Added more detailed module information:

```
MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("Missile launcher for CPRE488 MP-3");
MODULE_AUTHOR("Eastburn, Ohnesorge");
```

Structure and Naming Changes

Renamed all primary structures and functions:

- struct usb_skel → struct usb_miss_launch
- All function prefixes: skel → *miss_launch*
- Driver name: "skeleton" → "missile_launcher"
- Device naming: "skel%d" → "miss_launch%d"

Modified device structure:

- Replaced bulk_in_urb/bulk_out_endpoint with int_in_urb/int_in_endpoint
  - However, this int_in_endpoint was not used in the end since we only needed to send commands to endpoint 0, which is the default control endpoint.
- Removed numerous fields related to bulk transfer processing

Functional Changes

Read functionality

Original had complex logic for reading data via bulk transfers.
New driver simplifies to immediately return 0 (no reads).

Write functionality

- Completely redesigned to use control messages instead of bulk transfers

- Added missile launcher specific command formatting:

Uses an 8-byte fixed command buffer (LAUNCHER_CTRL_BUFFER_SIZE)
Sets byte 0 to command prefix (0x2)
Sets byte 1 to the actual command from user
Uses launcher-specific control message parameters

USB communication method:

Original used URBs for bulk data transfer
New driver uses usb_control_msg() with specific parameters:

```
usb_control_msg(dev->udev,
                usb_sndctrlpipe(dev->udev, 0),
                LAUNCHER_CTRL_REQUEST,
                LAUNCHER_CTRL_REQUEST_TYPE,
                LAUNCHER_CTRL_VALUE,
                LAUNCHER_CTRL_INDEX,
                command_buf,
                command_size,
                1000);
```

Endpoint detection

Original looked for bulk-in and bulk-out endpoints
New driver looks for interrupt-in endpoints, even though it is not used. This can be removed!

Memory Management

The new driver:

- Uses simple kmalloc instead of usb_alloc_coherent
- Allocates fixed-size buffers for commands
- Explicitly zeroes buffer memory with memset
- Implements proper cleanup with separate flags for memory and semaphore management

Error Handling and Resource Management

Simplified `miss_launch_draw_down()`:

- Original had complex handling for multiple URBs
- New driver simply kills the interrupt URB

Improved exit handling:

Added flags to track resource allocation states
Ensures proper cleanup in all error cases

Resource Protection

In both drivers, a semaphore called limit_sem is initialized with WRITES_IN_FLIGHT (set to **8**):
`sema_init(&dev->limit_sem, WRITES_IN_FLIGHT);`

Key Functions of the Semaphore

Memory Exhaustion Prevention

By limiting concurrent USB operations, the semaphore prevents excessive memory allocation that could otherwise exhaust system resources, particularly important in kernel space where memory is limited.

Flow Control

The semaphore regulates the flow of data from userspace to the USB device, preventing overwhelming the device with too many simultaneous commands.

I/O Mode Support

The implementation handles both blocking and non-blocking I/O modes:

In blocking mode: Uses down_interruptible() which may put the process to sleep
In non-blocking mode: Uses down_trylock() which returns immediately if the semaphore can't be acquired

Implementation Improvement

The missile launcher driver improves upon the original skeleton driver by adding explicit tracking of semaphore acquisition:
`int sem_downed = 1; // Flag to track if semaphore was acquired`
And later ensuring proper cleanup:

```
1  exit:
2      if(sem_downed)
3      {
4          up(&dev->limit_sem);
5      }
```

This prevents potential semaphore leaks in error conditions, which could otherwise lead to deadlocks where subsequent write operations would be permanently blocked after an error

occurred.

Operation of Initial `launcher_fire.c`

The code defines a set of command constants that correspond to different actions the missile launcher can perform, such as:

- LAUNCHER_FIRE (`0x10`): Fires a dart/missile
- LAUNCHER_STOP (`0x20`): Stops all movement
- LAUNCHER_UP (`0x02`), DOWN (`0x01`), LEFT (`0x04`), RIGHT (`0x08`): Directional controls
- Combined directions like UP_LEFT, DOWN_RIGHT, etc.

The program relies on the Linux character device driver properly translating these simple commands into the appropriate USB control messages that the missile launcher hardware can understand.

The main logic is implemented through:

The `launcher_cmd()` function which:

1. Takes a file descriptor and command as input
2. Attempts to write the command to the device file
3. Handles error conditions if the write fails
4. Adds a 5-second delay after firing to allow the launcher to complete its firing sequence

The `main()` function which:

1. Opens the device file `/dev/launcher0` with read/write permissions
2. Sets the command to LAUNCHER_FIRE (this is fixed, not user-controlled)
3. Sends the fire command to the launcher
4. Waits for a specified duration (500ms)
5. Sends the stop command
6. Closes the file descriptor

This particular functionality is quite simple - it just fires the missile launcher once when executed.

Algorithm to detect Target

System Architecture

The system combines computer vision processing with physical hardware control, structured around several key components:

1. **Framebuffer Interface**: The system maps the FPGA's framebuffer memory directly into the application's memory space, providing direct access to the camera feed.

```
// Map framebuffer memory
fb_mem = mmap(NULL, FB_SIZE, PROT_READ, MAP_SHARED, mem_fd, FB_PHYS_ADDR);
```

2. **Launcher Control Interface**: A device driver interface allows the software to command a physical launcher device through simple directional commands.
3. **Computer Vision Processing**: The system uses OpenCV for real-time target detection, combining color-based and shape-based detection methods.
4. **Targeting Logic**: Once targets are detected, the system performs depth estimation and aims the launcher accordingly.

Target Detection Algorithm

Sub-Group B's Implementation

NOTE: Since we could not get Petalinux to work with OpenCV (ran into boot issues), this implementation was not functional on the Zedboard. However, we have ran this implementation on a laptop with its webcam and it works as expected. Refer to sub-group A's implementation for grading as it is functional on the Zedboard.

The detection algorithm employs a dual-method approach to maximize reliability:

1. Color-Based (HSV) Detection

The primary detection method uses HSV color filtering to isolate potential targets based on their color:

```
cv::Point detect_target_hsv(cv::Mat &frame,
                            cv::Mat &debug_frame,
                            DetectionParams &params,
                            bool &detected,
                            float &estimated_z) {
  // Convert the frame from BGR to HSV color space
```

```
  cv::Mat hsv_frame;
  cv::cvtColor(frame, hsv_frame, cv::COLOR_BGR2HSV);

  // Create binary masks for the selected color range
  cv::Mat mask1, mask2, mask;
  cv::inRange(hsv_frame, params.primary.lower, params.primary.upper,
mask1);

  if (params.useMultiRange) {
    cv::inRange(hsv_frame, params.secondary.lower, params.secondary.upper,
mask2);
    cv::bitwise_or(mask1, mask2, mask);
  } else {
    mask = mask1;
  }

  // Apply morphological operations to clean up the mask
  // ...
}
```

The algorithm accommodates complex color ranges (like red, which wraps around the hue spectrum) by using multiple threshold ranges when needed.

2. Shape-Based Detection

As a fallback, the system also implements shape-based detection using both Hough Circle Transform and contour circularity analysis:

```
// Use Hough Circle Transform to detect circles
std::vector<cv::Vec3f> circles;
cv::HoughCircles(gray, circles, cv::HOUGH_GRADIENT, 1,
                 gray.rows / 8, 100, 30, 10, 100);
```

For enhanced reliability, the system also analyzes contour circularity when Hough transform fails:

```
// Circularity = 4π × Area / Perimeter²
// Perfect circle has circularity = 1
double circularity = 4 * M_PI * area / (perimeter * perimeter);

if (circularity > 0.7 && circularity > maxCircularity) {
  maxCircularity = circularity;
  bestContour = i;
}
```

Depth Estimation

A key feature of this system is the ability to estimate the target's distance from the camera based on its apparent size (knowing the camera's focal length, target diameter, and apparent diameter):

```
float estimate_z_position(double apparent_diameter_pixels) {
  if (apparent_diameter_pixels <= 0) {
    return MAX_TARGET_DISTANCE_CM;
  }

  float estimated_distance = (FOCAL_LENGTH_PIXELS *
TARGET_ACTUAL_DIAMETER_CM) / apparent_diameter_pixels;

  if (estimated_distance < MIN_TARGET_DISTANCE_CM) {
    estimated_distance = MIN_TARGET_DISTANCE_CM;
  } else if (estimated_distance > MAX_TARGET_DISTANCE_CM) {
    estimated_distance = MAX_TARGET_DISTANCE_CM;
  }

  return estimated_distance;
}
```

This calculation uses the principle that a target of known physical size (`TARGET_ACTUAL_DIAMETER_CM`) appears smaller in the image as its distance increases.

By knowing the camera's focal length (converted to pixels), we can derive the distance.

Launcher Control Algorithm

The launcher control system uses a proportional approach to aim at targets:

```
int aim_launcher(int launcher_fd,
                 int current_x,
                 int current_y,
                 int target_x,
                 int target_y,
                 float target_z) {
  int dx = target_x - current_x;
  int dy = target_y - current_y;

  // If target is already centered (within dead zone), we're done
  if (abs(dx) < LAUNCHER_DEAD_ZONE && abs(dy) < LAUNCHER_DEAD_ZONE) {
    // Apply depth-based adjustments
    ret = adjust_aim_for_depth(launcher_fd, target_z);
```

```
    return ret;
  }

  // Handle horizontal movement first
  // ...
}
```

Notable aspects of the aiming algorithm:

1. It implements a dead zone to prevent jitter when the target is nearly centered
2. It handles horizontal and vertical movements separately
3. Movement duration is proportional to the distance the launcher needs to travel
4. It includes a recursive approach to refine aiming with a limit of 5 attempts
5. It adjusts aim based on target depth to account for projectile ballistics

```
int adjust_aim_for_depth(int launcher_fd, float target_z) {
  // Skip adjustment for close targets
  if (target_z <= 100.0f) {
    return 0;
  }

  // Calculate adjustment time - more adjustment for distant targets
  int adjustment_ms = (int)((target_z - 100.0f) * 0.5f);

  if (adjustment_ms > 0) {
    std::cout << "Depth adjustment: Moving UP for " << adjustment_ms
              << " ms to compensate for distance" << std::endl;
    move_launcher(launcher_fd, LAUNCHER_UP);
    delay_ms(adjustment_ms);
    stop_launcher(launcher_fd);
    return 0;
  }

  return 0;
}
```

Firing Logic

The system employs a confirmation-based firing mechanism to prevent false positives:

```
if (target_detected) {
  std::cout << "Target detected at position (" << target_point.x
```

```
            << ", " << target_point.y << ", " << target_z
            << " cm)" << std::endl;

    consecutive_detections++;

    // Only fire when we have consistent detections to avoid false positives
    if (consecutive_detections >= fire_threshold && fire_cooldown <= 0) {
      // Aim launcher at the target
      ret = aim_launcher(launcher_fd, LAUNCHER_CENTER_X,
                         LAUNCHER_CENTER_Y, target_point.x,
                         target_point.y, target_z);

      if (ret == 0) {
        std::cout << "Target locked, firing!" << std::endl;
        // Fire the launcher
        fire_launcher(launcher_fd);

        // Set cooldown period after firing
        fire_cooldown = 20; // Approx 2 seconds at 100ms loop time
        consecutive_detections = 0;
      }
    }
  }
```

Key aspects of the firing logic:

1. It requires multiple consecutive successful detections before firing
2. It implements a cooldown period after firing to prevent rapid repeated firing
3. It only fires after the aiming process has completed successfully

Configurable Parameters

```
// Target recognition parameters
#define MIN_CONTOUR_AREA 500
#define MAX_CONTOUR_AREA 50000
#define MORPH_KERNEL_SIZE 5

// Launcher control parameters
#define LAUNCHER_MOVE_TIMEOUT_MS 1000
#define LAUNCHER_CENTER_X (FB_WIDTH / 2)
#define LAUNCHER_CENTER_Y (FB_HEIGHT / 2)
#define LAUNCHER_DEAD_ZONE 20
#define LAUNCHER_MAX_X_ANGLE 30
#define LAUNCHER_MAX_Y_ANGLE 20
```

```
13
14  // Z-position (depth) estimation parameters
15  #define TARGET_ACTUAL_DIAMETER_CM 15.0
16  #define CAMERA_FOV_HORIZONTAL_DEG 60.0
```

Error Handling

Added error handling helps to manage potential issues with hardware interfaces:

```
1   mem_fd = open("/dev/mem", O_RDWR | O_SYNC);
2   if (mem_fd < 0) {
3     perror("Failed to open /dev/mem");
4     return -1;
5   }
6
7   // Map framebuffer memory
8   fb_mem = mmap(NULL, FB_SIZE, PROT_READ, MAP_SHARED, mem_fd, FB_PHYS_ADDR);
9   if (fb_mem == MAP_FAILED) {
10    perror("Failed to mmap framebuffer");
11    close(mem_fd);
12    return -1;
13  }
```

Sub-Group A's Implementation

This is a working implementation

Color-based detection:

This approach uses entirely color-based detection to determine the target. It uses preset values to determine the intended target.
Once detected a target pixel it adds the information to the target variable which holds the center of all pixels found

```
1   int detect_target(uint16_t (*frame)[1920], Target* target) {
2       uint64_t x_sum = 0, y_sum = 0;
3       uint32_t count = 0;
4
5       // Scan entire frame for target color
6       for (int y = 0; y < 1080; y++) {
7           for (int x = 0; x < 1728; x += 2) {
8               uint16_t pixel1 = frame[y][x];
9               uint16_t pixel2 = frame[y][x+1];
10
```

```
            // Extract YUV components
            uint8_t y1 = pixel1 & 0xFF;
            uint8_t u = (pixel1 >> 8) & 0xFF;
            uint8_t y2 = pixel2 & 0xFF;
            uint8_t v = (pixel2 >> 8) & 0xFF;

            // Detect target color
            if (y1 > TARGET_Y_MIN && y1 < TARGET_Y_MAX && u > TARGET_U_MIN
&& u < TARGET_U_MAX && v < TARGET_V_MAX && v > TARGET_V_MIN) {
                x_sum += x;
                y_sum += y;
                count++;
            }
        }
    }

    // Minimum pixel threshold
    if (count > 1000) {
        target->x = x_sum / count;
        target->y = y_sum / count;
        target->count = count;
        return 1;
    }
    return 0;
}
```

Preset values:

```
// Detection parameters - edit for a different color(red)
#define TARGET_Y_MIN    38
#define TARGET_Y_MAX    47
#define TARGET_U_MIN    117
#define TARGET_U_MAX    123
#define TARGET_V_MIN    137
#define TARGET_V_MAX    144
```

Aim and fire

Once a target is detected, the code then moves the camera in line with the target and fires if the center matches the target

```
void aim_and_fire(int fd, Target* target) {
    const uint32_t center_x = 1920 / 2;
    const uint32_t center_y = 1080 / 2;
```

```
4
5       int x_offset = (int)target->x - (int)center_x;
6       int y_offset = (int)target->y - (int)center_y;
7
8       // Movement
9       if (x_offset > 50) {
10          launcher_cmd(fd, LAUNCHER_RIGHT);
11          usleep(abs(x_offset) * 50);
12          launcher_cmd(fd, LAUNCHER_STOP);
13          //printf("Center: RIGHT");
14      }
15  ...
16
17      // Fire if centered
18      if (abs(x_offset) < 50 && abs(y_offset) < 50) {
19          launcher_cmd(fd, LAUNCHER_FIRE);
20          usleep(2000000); // Wait for firing to complete
21      }
22
```